In **NumPy**, `copy()` is used to create a **new array with its own data**, independent of the original array.

Basic idea

```
import numpy as np

a = np.array([1, 2, 3])
b = a.copy()
```

- `a` and `b` have **the same values**
- but they **do not share memory**

Changing one does **not** affect the other.

```
b[0] = 100
print(a)  # [1 2 3]
print(b)  # [100 2 3]
```

Why `copy()` matters

Many NumPy operations return **views**, not copies.
A *view* shares the same underlying data.

Example: view vs copy

```
a = np.array([1, 2, 3, 4])
b = a[:2]        # view
c = a[:2].copy() # copy

b[0] = 99
print(a)  # [99  2  3  4]

c[1] = 88
print(a)  # [99  2  3  4]  (unchanged)
```

- `b` is a **view** → modifying it changes `a`
- `c` is a **copy** → modifying it does NOT change `a`

Memory behavior

```
a = np.array([1, 2, 3])
b = a.copy()

a.base is None      # True
b.base is None      # True
```

- `copy()` allocates **new memory**
- `base` is `None` because it does not depend on another array

Shallow vs deep copy?

For NumPy arrays:

- `copy()` is effectively a **deep copy of the data buffer**
- For object arrays (`dtype=object`), the objects themselves are **not copied**, only the references are

```
a = np.array([[1], [2]], dtype=object)
b = a.copy()
b[0][0] = 99
print(a)  # [[99], [2]]
```

Copy vs assignment

```
a = np.array([1, 2, 3])
b = a        # no copy
c = a.copy() # real copy
```

Operation	Shares memory?	Safe to modify?
`b = a`	✅ Yes	❌ No
`a[:]`	✅ Yes (view)	❌ No
`a.copy()`	❌ No	✅ Yes

When should you use `copy()`?

Use `copy()` when:

- You need to modify an array **without affecting the original**
- You are unsure whether an operation returns a view
- You are passing arrays to functions that may mutate them

1. `array.copy()` VS `np.copy()`

They do **the same thing**.

```
b = a.copy()
b = np.copy(a)
```

- Both create a **new array with its own data**
- Both break memory sharing
- `array.copy()` is more common and idiomatic

---

2. `copy()` VS `deepcopy()` (from `copy` module)

`copy.copy()`

- Shallow copy
- **Not recommended** for NumPy arrays

`copy.deepcopy()`

- Recursively copies Python objects
- **Overkill** for numeric NumPy arrays

```
import copy

a = np.array([1, 2, 3])
b = copy.deepcopy(a)
```

This behaves like `a.copy()` but is:

- slower
- unnecessary for numeric arrays

⚠️ **Exception**: object arrays

```
a = np.array([[1], [2]], dtype=object)
b = a.copy()
c = copy.deepcopy(a)

b[0][0] = 99
print(a)  # [[99], [2]]

c[1][0] = 77
print(a)  # unchanged
```

- `a.copy()` copies references

- `deepcopy()` copies the actual objects

---

3. How to check if arrays share memory

Using `np.shares_memory`

```
np.shares_memory(a, b)
```

Returns `True` or `False`.

Using `.base`

```
b = a[1:]
print(b.base is a)  # True
```

⚠️ `.base` only detects **direct** views
`np.shares_memory()` is more reliable.

---

4. Common operations that return views (⚠️ no copy)

These **share memory**:

```
a[1:5]
a[:, 0]
a.reshape(...)
a.T
```

Example:

```
b = a.reshape(2, 2)
b[0, 0] = 999
```

`a` changes too.

---

5. Operations that return copies

These usually **allocate new memory**:

```
a.copy()
a + 1
```

```
a * 2
np.array(a)
a.astype(float)
```

⚠️ Note: `np.array(a)` may return a view if `copy=False` is allowed.

---

6. Performance implications

Copying large arrays is **expensive**:

- Time: O(n)
- Memory: duplicates the full buffer

```
# Avoid unnecessary copies
b = a[a > 0]      # copy (boolean indexing)
c = a[:100]       # view (cheap)
```

Use views when:

- You only need to read data
- Memory is a concern

Use `copy()` when:

- You must modify safely

---

7. Rule of thumb (very useful)

If you plan to write to an array and you're not 100% sure it's a copy — call `.copy()`

---

8. Visual summary

```
Assignment        → no copy
Slicing           → view
Boolean indexing  → copy
Arithmetic ops    → copy
.reshape(), .T    → view
.copy()           → copy
```